FIRST:

First(α) = ?

$\alpha = xyz$ ise First(α) = x eğer $x \in V_T$

değilse:

First(α) = First(x) eğer $E \notin$ first(x) (→ boşluk)

Eğer $E \in$ First(x) ise First(α) = [First(x) - E] ∪ First(yz)

First(yz) = y eğer $y \in V_T$

değilse: First(yz) = First(y) eğer First(y) E içermiyorsa

eğer $E \in$ First(y) ise First(yz) = [First(y) - E] ∪ First(z)

ÖRNEK:

$S \rightarrow ACB \mid CbB \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \Lambda$

$C \rightarrow h \mid \Lambda$

$V_T = \{a, b, d, g, h\}$

Bu gramerin first kümeleri?

- First(S) = first(ACB) ∪ first(CbB) ∪ first(Ba)
- First(A) = first(da) ∪ first(BC)
- first(B) = first(g) ∪ first(ε) = {g, ε}
- first(C) = first(h) ∪ first(ε) = {h, ε}
- first(BC) = first(B) ∪ first(C) = {g, h, ε}
- first(A) = {d, g, h, ε}
- first(ACB) = [first(A) - ε] ∪ first(CB)
  = {d, g, h} ∪ (first(C) - ε) ∪ first(B)
  = {d, g, h, ε}
- first(CbB) = (first(C) - ε) ∪ first(bB)
  = {h} ∪ {b} = {h, b}
- first(Ba) = (first(B) - ε) ∪ (first(a)) = {g, a}
- first(S) = {d, g, h, ε, b, a}

FOLLOW:

Kural: 1

S bir başlangıç NT ise

(EOS), $, ε

(EOS)'i follow kümesine ekle.

Kural: 2

$A \rightarrow \alpha B A$ iken

$B \rightarrow \varepsilon$ kuralı varsa

first(B)'deki tüm simgeleri (ε hariç)

follow(B)'ye ekle.

Kural: 3

$A \rightarrow \alpha B$

follow(B)'ye, follow(A) eklenir.

ÖRNEK:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

Bu gramerin follow kümeleri?

First(E) = First(T) = First(F) = { (, id }

First(E') = { +, ε }

First(T') = { *, ε }

Follow(E) = { $,) }

Follow(E') = follow(E)

Follow(T) = follow(T') = { +, $,) }

Follow(F) = { +, $,), * }

Derleyici T.

04.04.19

9. Hafta

-ÖRNEK-

$E \to TX$

$T \to (E) \mid int\ Y$

$X \to +E \mid \varepsilon$

$Y \to *T \mid \varepsilon$

Terminal ifadeler

Topdown parsing yap

- Sol rekürsiflik ve faktöritasyon kontrol edilir.
- ikisi de yok

LL(1)
- L: left to right
- L: leftmost ...
- (1: lookahead symbol) ileri bakış simgesi

- 1 simge okununca hangi kural çalışır belli?

E nonterminali çalışınca sağda alacak ilk simgeler.

1) FIRST(E) = {int, (}

FIRST(T) = {(, int}

FIRST(X) = {+, ε}

FIRST(Y) = {*, ε}

2) FOLLOW(E) = {$,)} ($: end of string)

FOLLOW(X) = FOLLOW(E) = {$,)}

FOLLOW(T) = FOLLOW(Y) = {+, follow(E)} = {+, $,)}

FOLLOW(Y) = {+, $,)}

	int	*	+	(	)	$
E	→TX	error	error	→TX	error	error
T	→int Y	error	error	→(E)	error	error
X	error	error	→+E	error	→ε	→ε
Y	error	→*T	→ε	error	→ε	→ε

→ FIRST kümelerinde yazılır, ε görünce X'i izleyenlere de bakılır. FOLLOW

Bu konu önemli ▽

— Sınavda çıkacak

- ÖRNEK 2 - (LL(1) gramer)

(Rekursive Descent) Tablosu:

1) Herbir terminal simge için ve end of string $ için bir sütun.
   Herbir nonterminal/terminal simge için ve (∇) için bir satır.
2) Her $A \to a\alpha$ kuralı için $(A \times a)$ hücresine $Rep(\alpha^R a)$ yaz.
3) $(a \times a)$, $(b \times b)$ → hücrelerine (pop/ilerle) yaz.
4) $(\nabla \times \hookleftarrow)$ hücresine (accept) yaz.
5) Her $A \to \varepsilon$ kuralı için A satırı ve follow(A) kümesindeki sütunlara pop/Retain yaz.
   (yığıttan sil ama pozisyonu koru?)
6) Diğer tüm sütunlara "Reject" yaz.
7) Yığıt 

başlangıçta yığıtı başlat.

Tanım

A çalıştı sonrakinin first'i

ÖRNEK

$S \to ABc$
$A \to bA$
$A \to \varepsilon$
$B \to c$

	b	c	↵
S	Rep(cBA) Retain	Rep(cBA) Retain	Reject
A	Rep(Ab)	pop Retain	Reject
B	Reject	Rep(c) Retain	Reject
b	pop ilerle	Reject	Reject
c	Reject	pop ilerle	Reject
∇	Reject	Reject	Accept

first(S) = {b, c}
first(A) = {b, ε}
follow(A) = {c}
first(B) = {c}

—ÖRNEK 3—

1) Expr → Expr + Term
2) Expr → Term
3) Term → Term * Factor
4) Term → Factor
5) Factor → (Expr)
6) Factor → var

Soru:
Bu gramer için topdown-parser tasarlayınız.

Hatırlatma:
Sol rekürsiflik:
$A \to A\alpha$
$A \to \beta$
} varsa.
yerine
$A \to \beta R$
$R \to \alpha R$
$R \to \varepsilon$ yazılır

④ kuralı yerine: Yeni nt
1) Expr → Term Elist
2) Elist → + Term Elist
3) Elist → ε
③ kuralı yerine:
4) Term → Factor TList
5) TList → * Factor TList
6) TList → ε
7) Factor → (Expr)
8) Factor → var

SORU: LL(1) midir?

Kural 1: first(Term Elist) = { (, var } = first(Expr)

first(Elist) = first(+ Term Elist) = { + } ∪ first(ε) = { +, ε }

follow(Elist) = {), ↵ }

first(Term) = first(factor Tlist) = { (, var }

first(TList) = first(* Factor TList) = { * }

follow(TList) = { +,), ↵ }

first(Factor) = { (, var }

→ Tablo

	+	*	(	)	var	↵
Expr	X	X	Rep(Elist Term) Retain	X	Rep(Elist Term) Retain	
Elist	Rep(Elist Term+) Retain	X	X	pop Retain	X	pop Retain
Term	X	X	Rep(Tlist Factor) Retain	X	Rep(Tlist Factor) Retain	X
Tlist	pop Retain	Rep(Tlist Factor*) Retain	X	pop Retain	X	pop Retain
+	pop ilerle	X	X	X	X	X
*	X	pop ilerle	X	X	X	X
(	X	X	pop ilerle	X	X	X
)	X	X	X	pop ilerle	X	X
var	X	X	X	X	pop ilerle	X
Factor	X	X	Rep( ) Expr ( )	X	Rep(Var) Retain	X
▽	X	X	X	X	X	Accept

X : Reject

Teisten yazıldığı için

Rep ((Expr))

Syntax Directed Translation:

A → {action} → çeviri grameri

Örnek:

G: S → {print} aS

S → bB

B → {print}

	a	b	↵
S	Rep(Sa {print} Retain	Rep(Bb) Retain	—
B	—	—	Rep({print}) Retain
a	pop aduance	—	—
b	—	pop aduance	—
{print}	pop Retain out({print})	pop retain out({print})	pop retain out({print})
▽	—	—	ACCEPT!




```
void S(){
 if (input==a)
    { cin>>inp;
      cout << " print";
      S();
    }
 else if (inp==b)
    { cin>>inp;
      B();
    }
 else Reject();
    }
Void B(){
   if (inp=='$')
   cout << " print";
   else Reject();
        }
```

kural 1

kural 2

kural 3

ÖRNEK:

1- Expr → TermElist

Elist → +Term {+} Elist

Elist → ε

Term → Factor TList

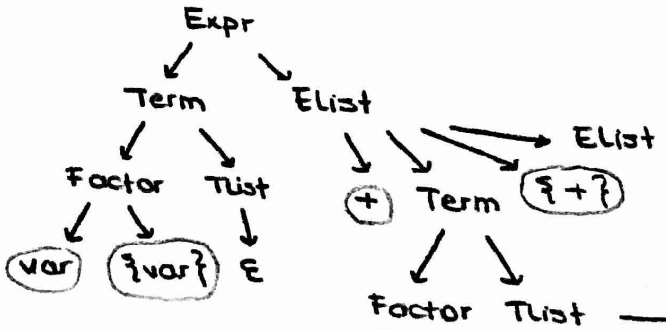
Tlist → *Factor {*} Tlist

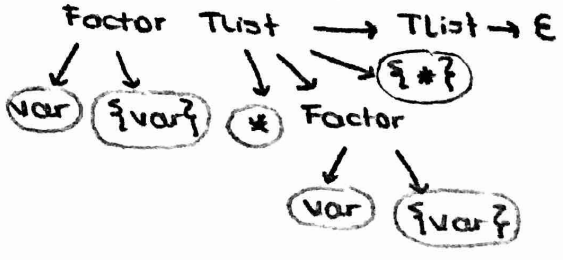
Tlist → ε

Factor → Expr

Factor → var {var}

var + var * var türer mi?



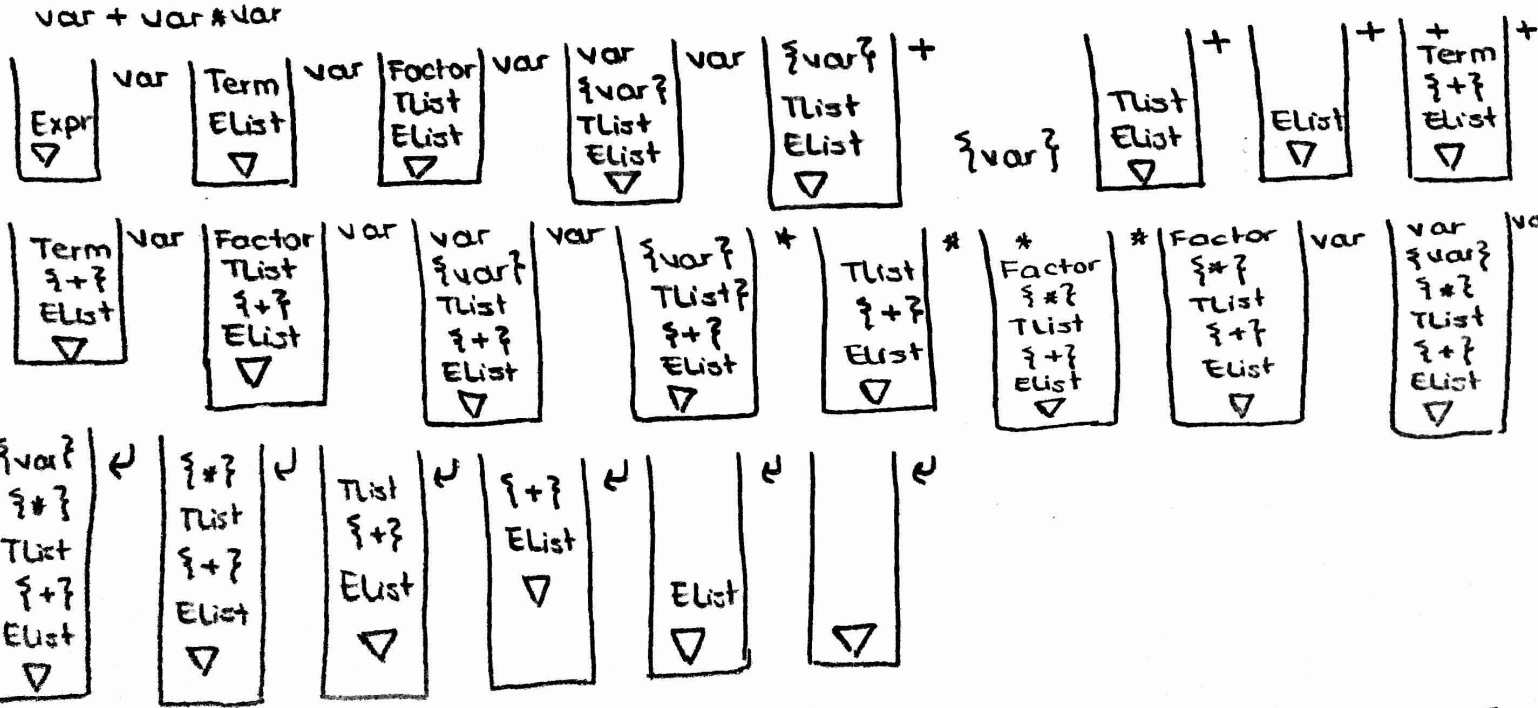


var {var} + var {var} * var {var} {*} {+}

{var} {var} {var} {*} {+}

	var	+	*	(	)	↵
Expr	Rep(EListTerm) Retain			Rep(EListTerm) Retain		
EList		Rep(EList{+}Term +} Retain			pop Retain	pop Retain
Term	Rep(TListFactor) Retain			Rep(TList Factor) Retain		
TList		pop Retain	Rep(TList{*} Factor* Retain			
Factor	Rep({var}var) Retain			Rep((Expr)) Retain		
Var	pop aduance					
+		pop aduance				
*			pop aduance			
(				pop aduance		
)					pop aduance	
{var}	pop Retain out(var)	out{var}				pop Retain Out{var}
{+}	pop Retain out(+)	out{+}				pop Retain out{+}
{*}	pop Retain Out(*)	out{*}				pop Retain out{*}
▽						ACCEPT!

var + var * var



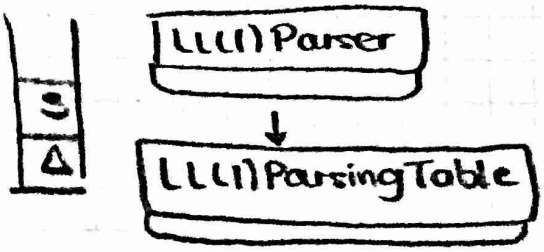
AKADEMİK BİLİŞİM 2019 ORDU ÜNİVERSİTESİ

25.04.19

11. Hafta

PARSER

- Topdown
  - non-deterministik backtracking
    - without backtracking deterministik Sol rekürsiyon x
      - LL(1) Parser
  - Bottom-up
    - shift-reduce parser



BOTTOMUP

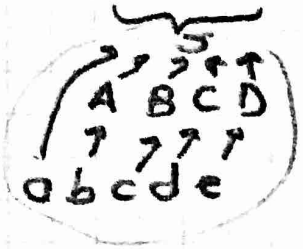
ÖRNEK:

	First	Follow
S → aABCD	{a}	$
A → b	{b}	{c}
B → c	{c}	{d}
C → d	{d}	{e}
D → e	{e}	$

S → aABCD → abBCD → abcCD → abcdD → abcde

ÖRNEK:

	First	Follow
S → ABCD	{b, Δ, c}	$
A → b | Δ	{b, Δ}	{c}
B → c	{c}	{d}
C → d	{d}	{e}
D → e	{e}	$



Bottom-up

NOT: Örneğin: A'nın follow'u, B'nin First'ü

	FIRST	FOLLOW
$S \to ABCDE$	$\{a, b, c\}$	$\$$
$A \to a / \Lambda$	$\{a, \Lambda\}$	$\{b, c\}$
$B \to b / \Lambda$	$\{b, \Lambda\}$	$\{c\}$
$C \to c$	$\{c\}$	$\{d, \$, e\}$
$D \to d / \Lambda$	$\{d, \Lambda\}$	$\{e, \$\}$
$E \to e / \Lambda$	$\{e, \Lambda\}$	$\$$

Follow (x)

- S başlangıç ise $ simgesini followa ekle.
- $A \to \alpha B \beta$ ise first(β) kümesinin Λ dışındaki tüm elemanlarını follow(B)'ye ekle. follow(B) = first(β)
- $A \to \alpha B$ veya $A \to \alpha B \beta$ ve $\Lambda \in$ first(β) ise follow(A) kümesinde yer alan tüm elemanları follow(B)'ye ekle.
- $A \to \alpha B$ veya $A \to \alpha B \beta$ $\beta \overset{*}{\to} \Lambda$ follow(B) = follow(A) ekle.

AKADEMİK BİLİŞİM 2019 ERÜ ÜNİVERSİTESİ

ÖRNEK:

S → ACB | CbB | Ba
A → da | BC
B → g | Λ
C → h | Λ

} FIRST ~ FOLLOW?

	FIRST	FOLLOW
S	{d, g, h, Λ, b, a}	$
A	{d, g, h, Λ}	{h, g, $}
B	{g, Λ}	{$, a, h, g}
C	{h, Λ}	{g, $, b, h}

ÖRNEK:

	FIRST	FOLLOW
S → aABb	{a}	$
A → c / Λ	{c, Λ}	{d, b}
B → d / Λ	{d, Λ}	{b}

ÖRNEK:

	FIRST	FOLLOW
S → aBDh	{a}	$
B → cC	{c}	{g, f, h}
C → bC / Λ	{b, Λ}	{g, f, h}
D → EF	{g, Λ, f}	{h}
E → g / Λ	{g, Λ}	{f, h}
F → f / Λ	{f, Λ}	{h}

→ C'yi kapsayanlar B ve C

B'nin follow'u C'nin de follow'u

	FIRST	FOLLOW
$E \to TE'$	$\{id, (\}$	$\{\$,)\}$
$E' \to +TE' \,/\, \Lambda$	$\{+, \Lambda\}$	$\{\$,)\}$
$T \to FT'$	$\{id, (\}$	$\{+,), \$\}$
$T' \to *FT' \,/\, \Lambda$	$\{*, \Lambda\}$	$\{+,), \$\}$
$F \to id \,/\, (E)$	$\{id, (\}$	$\{+,), \$, *\}$

E' = E followları eşit

T ve T' eşit + (*)

Follow bulurken → burada nerede var diye düşünülür

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to TE$			$E' \to \Lambda$	$E' \to \Lambda$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \Lambda$	$T' \to *FT'$		$T' \to \Lambda$	$T' \to \Lambda$
F	$F \to id$			$F \to (E)$		

E'den id'ye gitmek ? Yolu yazılır

LR(0), LR(1) için Ayrıştırıcı (Parser) NFA'nın Oluşturulması:

1) $S' \to S$ yeni bir NT(S') ve yeni bir kural ekle ve bunu 1 numaralı durum yap.

2) $A \to \ldots . X \ldots \; X \in (V_N \cup V_T)$

X geçişiyle $A \to \ldots X . \ldots$ hangi duruma giderim?

3) $A \to \ldots X.$ Herhangi bir düğümde "." biçiminde bir kural var ve bu kural o düğümdeki tek kural ise o düğüm bir kabul durumudur.

4) Yeni kural eklenmeyene kadar ve bütün kurallar sınanıncaya kadar bu işlemlere devam edilir.

PUSHDOWN AUTOMATA MARKED RULES:

$S \to aAb$ biçimli bir kural var ise; soldan türetime başlarsak

$S \to .aAb$ (işaretli kuralın ilk biçimi)

$S \to a.Ab$
$S \to aA.b$ } (ara biçimler)

$S \to aAb.$ (işaretli kuralın son biçimi)

İşaret Kural Kümesinin Kapanışı:

K kural kümesi üzerinden incelersek;

K^+ kümesine; K işaretli kurallar kümesinin kapanışı:

Eğer işaretten $^\bullet$ sonra NT varsa bu NT'in ilk biçimi kümeye eklenir. Yeni kural işlenemeyinceye kadar kural eklenmeye devam eder.

ÖRNEK:

$S \to aS \mid A$
$A \to Ab \mid B$
$B \to Bc \mid c$
$C \to Cc \mid c$

$K: \{S \to a.S,\ C \to .c\}$

K^+ (K kümesinin kapanışını bulunuz.)

K^+ : $S \to a.S$, $C \to .c$

$S \to .aS$, $S \to .A$

$A \to .Ab$, $A \to .B$

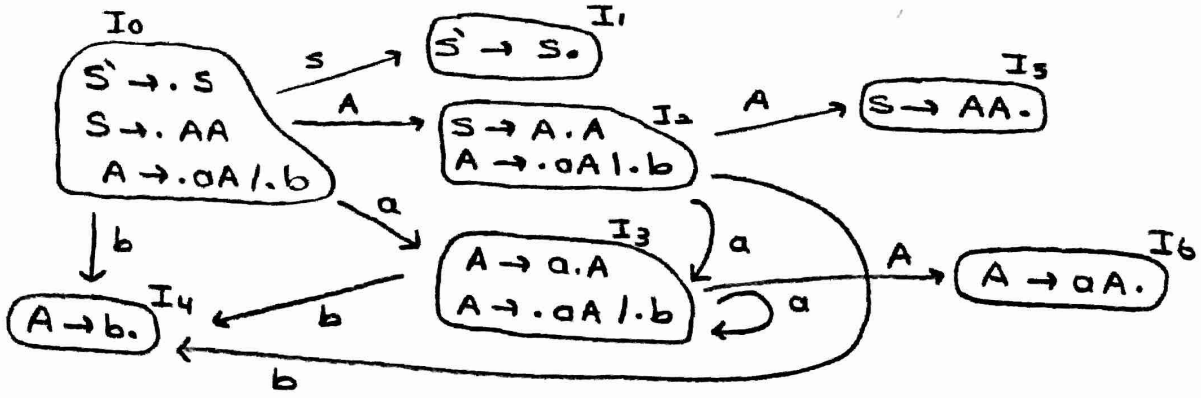
$B \to .Bc$, $B \to .C$

$C \to .Cc$

LR(0) PARSING TABLE

S → AA (1)
A → aA (2) | b (3)

S` → S
(1) S → AA
(2) A → aA | b (3)



	action				
	a	b	$	A	S
0	s_3	s_4		2	1
1			Accept		
2	s_3	s_4		5	
3	s_3	s_4		6	
4	$r_3 r_3$	$r_3 r_3$	$r_3 r_3$		
5	$r_1 r_1$	$r_1 r_1$	$r_1 r_1$		
6	$r_2 r_2$	$r_2 r_3$	$r_2 r_2$		

r'ler hangi kuraldan türedilerse.
Kabul durumları,,

Sağ taraf ise hangi durumdan NT gelirse hangi duruma gidilecek?

I_4 follow(A) = follow(S) ∪ follow(A)
I_6 follow(A) = follow(S) ∪ follow(A)
I_5 follow(S) = $

} devamı tabloda olmayan kabul durumlarının follow'ları.

KATAR TÜRETİMİ: aabb?

Stack	input	action
0	aabb$	–
$0a_3$	abb$	Shift 3
$0a_3a_3$	bb$	Shift 3
$0a_3a_3b_4$	b$	shift 4
$0a_3a_3A_6$	b$	reduce 3 (A → b)
$0a_3A_6$	b$	reduce 2 (A → aA)
$0A_2$	b$	reduce 2 (A → aA)
$0A_2b_4$	$	shift 4
$0A_2A_5$	$	reduce 3 (A → b)
$0S_1$	$	reduce 1 (S → AA)
	↘ accept	

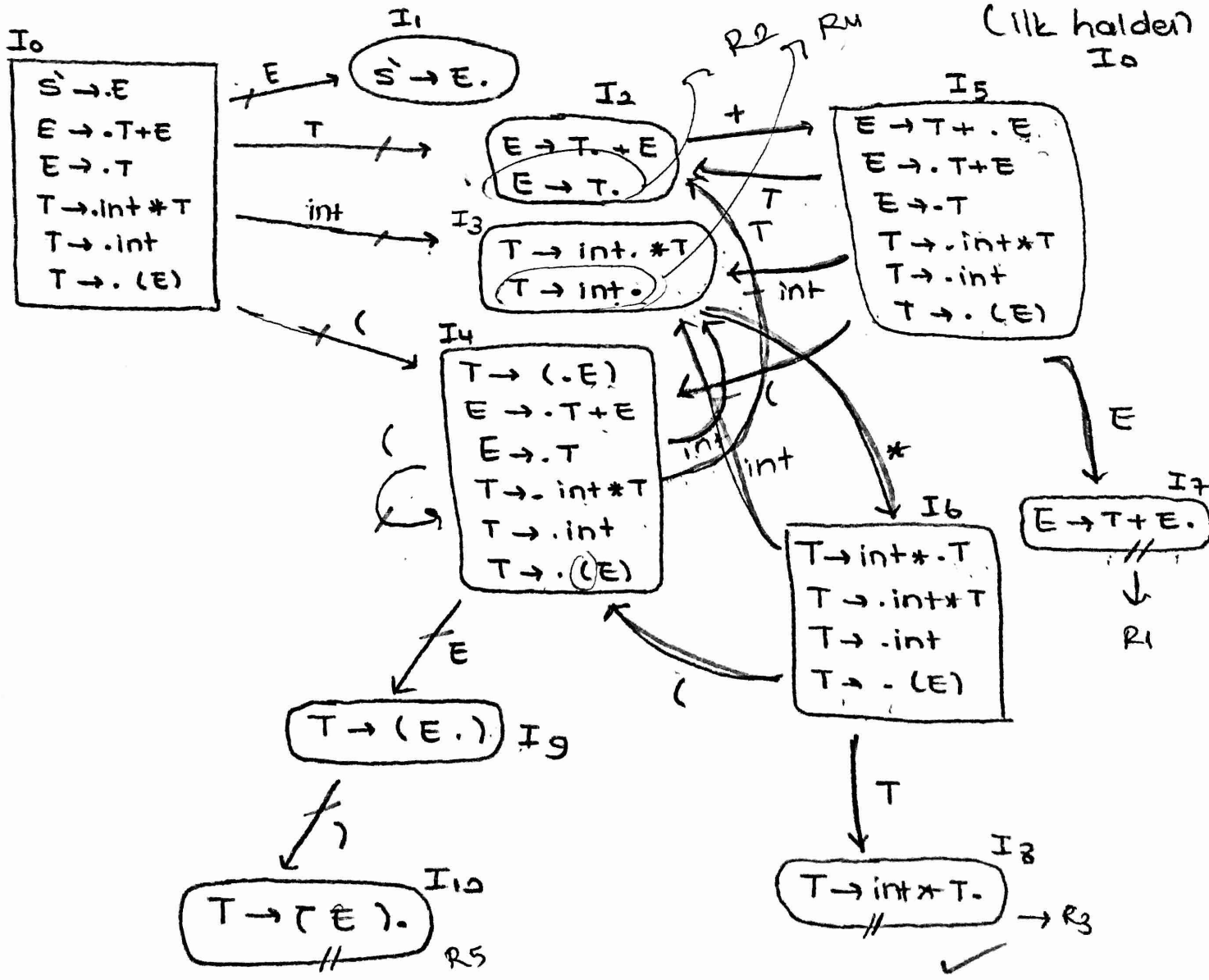
?
o

SORU

① E → T+E
② E → T
③ T → int * T
④ T → int
⑤ T → (E)

S` → .E
E → .T+E
E → .T
T → .int*T
T → .int
T → .(E)

Noktadan sonra NT varsa o NT'iden itibaren kuralları da ekle.



kabuller: $I_{10} - I_8 - I_7 - I_1$

follow(E) = { $,) }

follow(T) = { +, $,) }

Tablo

	int	(	)	+	*	$	E	T
0	S_3	S_4					1	2
1						Accept		
2			R_2	S_5		R_2		
3			R_4	R_4	S_6	R_4		
4	S_3	S_4					9	2
5	S_3	S_4					7	2
6	S_3	S_4						8
7			R_1			R_1		
8			R_3	R_3		R_3		
9			S_{10}					
10			R_5	R_5		R_5		

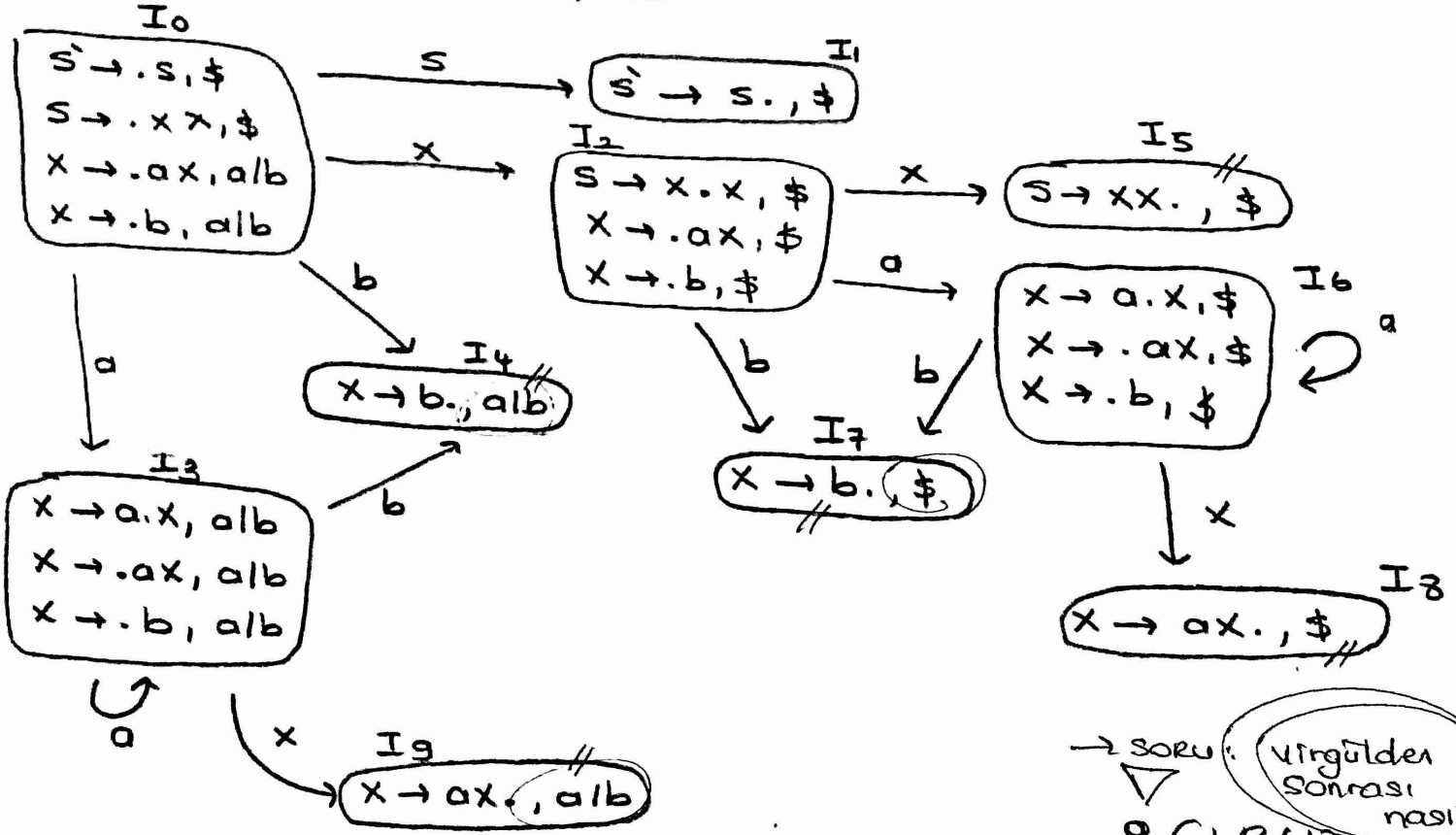
~ LR(0) ve SLR(1) Tablosu ~

CLR(1) , LALR(1)

S → XX
X → aX
X → b

S` → .S, $
① S → .XX, $
② X → .aX, a/b
③ X → .b, a/b

first(X) = {a, b}



→ SORU: Virgülden sonrası nasıl?

∇

● CLR(1) Tablosu

	a	b	$	S	X
0	S3	S4		1	2
1			accept		
2	S6	S7			5
3	S3	S4			9
4	r3	r3			
5			r1		
6	S6	S7			8
7			r3		
8			r2		
9	r1	r2			

follow(S) = {$}
follow(X) = {$, a, b}

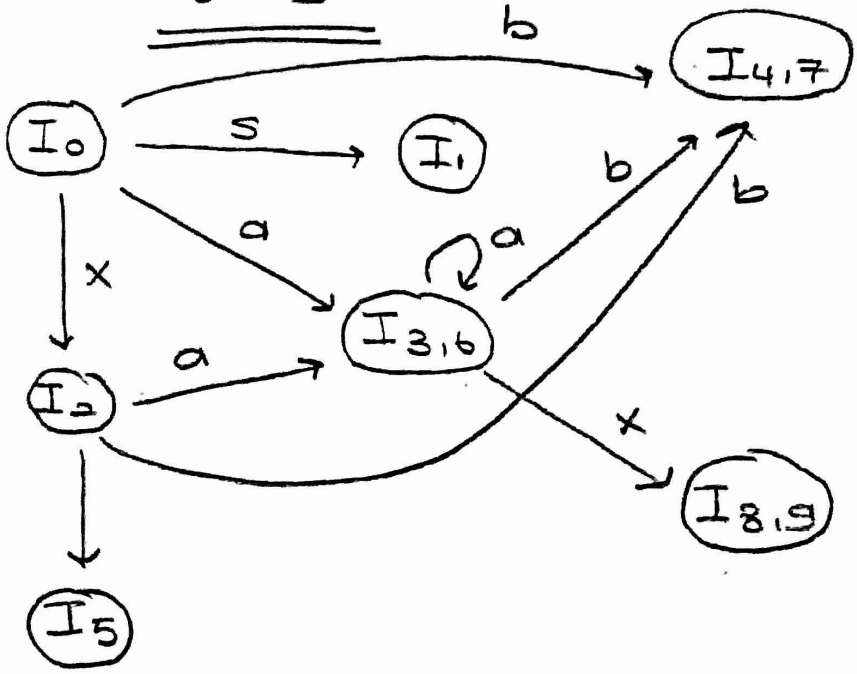
LARLR yaparken CLR tablosunda aynı kurallar birleştirilir.

Çizilen grafikteki kurallardan aynı olanlar:

3 - 6

4 - 7

8 - 9



	a	b	$	S	X
0	s_3	s_4		1	2
1				accept	
2	s_6	s_7			5
3,6	$s_{3,6}$	$s_{4,7}$			8,9
4,7	r_3	r_3	r_3		
,5			r_1		
8,9	r_2	r_2	r_2		

↓ CLR(1)'de Birleştirilip yazılır.

Satır sayıları için: CLR(1) > LR(0) = LALR(1) = SLR(1)

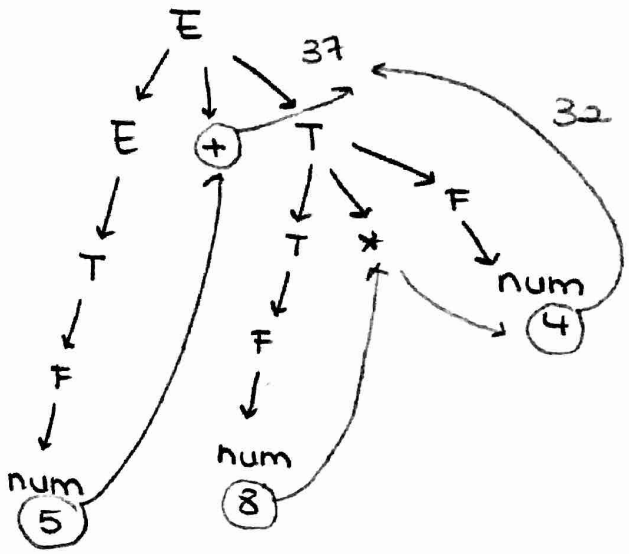
SDT

$E \to E + T \mid T$ {E.value = E.value + T.value} {E.value = T.value}

$T \to T * F \mid F$ {T.value = T.value * F.value} {T.value = F.value}

$F \to num$

5 + 8 * 4



$E \to E + T \ (1) \mid T \ (2)$

$T \to T * F \ (3) \mid F \ (4)$

$F \to num \ (5)$

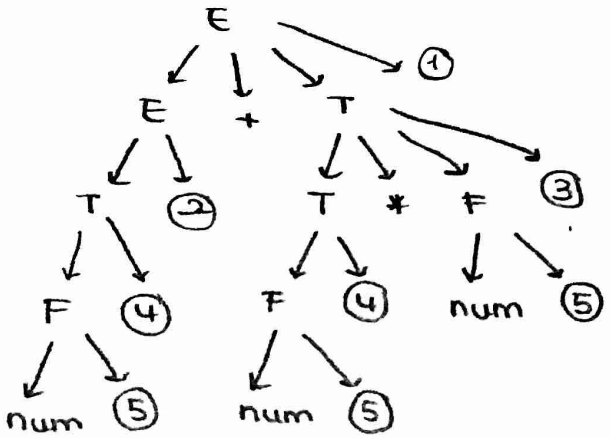
{print("+")}

{print("*")}

{print(num.val)}

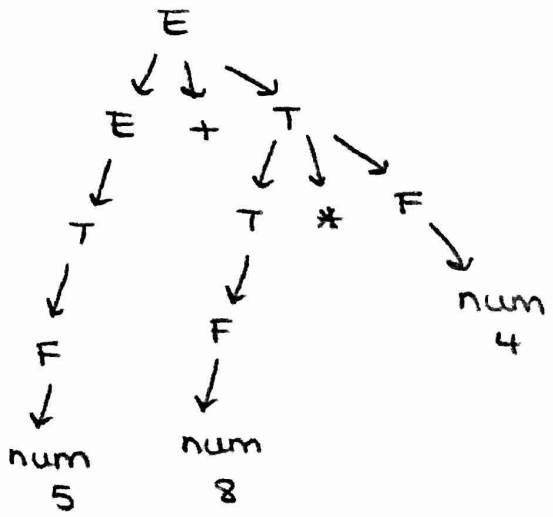
5 + 8 * 4

Topdown LL(1)



5 8 4 * T?

Bottom-Up



5 8 4 * + $